Volume V

Software Architecture & The 5-Block Invariant Protocol

CreativeWorkzStudio LLC / ProfessorSeanEX

Version 1.0.0, 2026-11-07

Table of Contents

1. Volume Synopsis . . . 1
2. Chapter 1: OmniCode 5-Block Architectural Framework . . . 2
3. Architecture Overview . . . 3
4. 1. The Universal 5-Block Code Structure . . . 4
5. 2. Core Mechanics & State Invariants . . . 5
    5.1. The Twin Relics: Shield and Sword . . . 5
    5.2. Invariant Axioms . . . 5
6. 3. Multi-Phase Engine Evolution . . . 6
7. 4. Layer 2 Presentation Architecture & 5-Screen Navigation . . . 7
8. Chapter 2: Dual-Frontend Presentation Layer (CLI + Native X11) . . . 8
9. Abstract . . . 9
10. 1. The Core Architectural Philosophy . . . 10
    10.1. Universal Action Dispatcher & Intent Emitter . . . 10
11. 2. Native Desktop GUI Architecture: Why Not Web? . . . 12
    11.1. Double-Buffering & Render Synchronization . . . 12
    11.2. Dynamic Visual Palette: The Twin Relics . . . 13
12. 3. The 5-Block Implementation Mapping . . . 14
    12.1. Presentation Architecture Breakdown (`include/gui/gui.h` & `src/gui/gui.c`) . . . 14
13. 4. Headless CI & Deterministic Replay . . . 15
14. 5. Custom Engine Evolution Roadmap . . . 16
15. Chapter 3: Central Manifest & Pre-Flight Diagnostic Integrity . . . 17
16. Specification Abstract & Purpose . . . 18
17. 1. Expected Architecture: The Central Manifest Pattern . . . 19
    17.1. 1.1 Structural Hierarchy . . . 19
    17.2. 1.2 The Diagnostic Severity Spectrum . . . 19
    17.3. 1.3 Expected Pre-Flight Engine States . . . 20
18. 2. Architectural Metadata Dimensions & Pedagogical Balance . . . 21
    18.1. 2.1 Striking the Balance: Beginner-Accessible yet Senior-Rigorous . . . 21
    18.2. 2.2 Explicit Temporal Stamping & Version History . . . 21
    18.3. 2.3 The 5-Dimension Compiled Schema (`module_metadata_t`) . . . 21
    18.4. 2.4 Active Dependency Graph Enforcement . . . 22
19. 3. Expected Pre-Flight "Loose Bolts" System Checks . . . 23
20. 4. Target Verification Criteria ("Expected" Baseline) . . . 24
21. 5. Actual Results vs. Expected Gap Analysis . . . 25
    21.1. 5.1 Empirical Benchmark Telemetry . . . 25
    21.2. 5.2 How to Reproduce and Verify Every Result . . . 26
    21.3. 5.3 Gap Analysis & Next Architectural Horizons . . . 27
22. Chapter 4: Layer 0 Core Utilities Code Tour . . . 28

23. Overview . . . 29
24. 1. Layer 0 Directory Strata & Subsystem Architecture . . . 30
25. 2. Filesystem & JSON Engines (`util/data/`) . . . 31
    25.1. 2.1 Filesystem Utilities (`fs_utils.h` & `fs_utils.c`) . . . 31
    25.2. 2.2 JSON Utilities (`json_utils.h` & `json_utils.c`) . . . 31
26. 3. Mathematical Invariant Utilities (`util/math/`) . . . 33
    26.1. 3.1 Bounded Arithmetic (`math_utils.h` & `math_utils.c`) . . . 33
    26.2. 3.2 2D Geometry Utilities (`geom_utils.h` & `geom_utils.c`) . . . 33
27. 4. Lexical & String Utilities (`util/string/`) . . . 34
28. 5. Monotonic Timing Utilities (`util/time/`) . . . 35
29. 6. Logging & Telemetry Subsystem (`util/log/`) . . . 36
30. 7. Central Manifest & Pre-Flight Registry (`manifest.h` & `manifest.c`) . . . 37
    30.1. 7.1 Pre-Flight Verification Telemetry . . . 37
31. Chapter 5: Layer 0 Debugging System & State Rewind . . . 38
32. 1. Architectural Philosophy & Mission . . . 39
    32.1. Key Architectural Pillars . . . 39
33. 2. Subsystem Layout & Strata . . . 40
34. 3. Core Mechanics & Invariants . . . 41
    34.1. 3.1 Frozen State Snapshot Ring Buffer . . . 41
    34.2. 3.2 Dynamic Breakpoints & Tripwire Traps . . . 41
    34.3. 3.3 State Differential Analysis (`debug_diff_snapshots`) . . . 41
35. 4. Interactive CLI Integration . . . 43
36. 5. Verification & Audit Contract . . . 44
37. Chapter 6: Layer 1 Simulation Engine Code Tour . . . 45
38. Overview . . . 46
39. 1. State Structure & Memory Layout (`player_state_t`) . . . 47
40. 2. Universal Action Dispatcher (`game_process_command`) . . . 48
    40.1. Execution Pipeline: . . . 48
41. 3. Diagnostic Self-Auditing (`game_audit_state`) . . . 49
42. Chapter 7: Layer 2 Presentation Viewport Code Tour . . . 50
43. Overview . . . 51
44. 1. The Modular GUI Sub-Assembly Architecture . . . 52
45. 2. Desktop Panel Zoning & Layout Invariants . . . 53
46. 3. The 2.5D Tactical Isometric Engine . . . 54
    46.1. Isometric Mathematical Formulation: . . . 54
    46.2. Unprojection Reciprocity: . . . 54
    46.3. 3D Prism Shading & Elevation Blocks: . . . 54
    46.4. Painter’s Algorithm Depth Sorting: . . . 54
47. 4. The Asset Abstraction Layer . . . 56
48. 5. Verification Telemetry . . . 57
49. Chapter 8: Layer 4 Verification & Test Suite Code Tour . . . 58

50. Overview . . . 59
51. 1. The Four-Tier Testing Hierarchy . . . 60
51.1. Tier 1: Functional Debugging (`tests/functional/`) . . . 60
51.2. Tier 2: Subsystem Unit Harnesses (`tests/unit/`) . . . 60
51.3. Tier 3: Integrated Utilities Harness (`tests/integrated/`) . . . 61
51.4. Tier 4: System Verification (`tests/system/`) . . . 61
52. 2. The Four Ontological Testing Sections . . . 62
53. 3. CMake & CTest Test Targets . . . 63
54. 4. Execution & Verification Commands . . . 64
55. Chapter 9: Technical Specification: Data-Driven Architecture & Manifest System . . . 65
56. Abstract & Architectural Mandate . . . 66
57. 1. System Philosophy: Manifest as Main Menu . . . 67
57.1. 1.1 The Menu-to-Settings Ontological Pattern . . . 67
57.2. 1.2 OmniCode 5-Block Conformance in JSON Serialization . . . 67
58. 2. Active Data-Driven Subsystems . . . 69
58.1. 2.1 Logging & Telemetry (`data/log/`) . . . 69
58.2. 2.2 Mathematical Continuum (`data/math/`) . . . 69
58.3. 2.3 Monotonic Timing & Frame Pacing (`data/time/`) . . . 69
58.4. 2.4 Lexical Buffers & Token Dictionary (`data/string/`) . . . 69
59. 3. The Fallback Tripwire Protocol . . . 70
60. 4. Verification & Diagnostic Testing . . . 71

Chapter 1. Volume Synopsis

System boundaries, layered strata, dual-frontend GUI/CLI, and pre-flight zero-heap diagnostic manifests.

Chapter 2. Chapter 1: OmniCode 5-Block Architectural Framework

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/ARCHITECTURE.adoc

Chapter 3. Architecture Overview

This document specifies the technical design, subsystem boundaries, data models, and ontological lifecycle governing **Tenebris and Lux** ("Darkness and Light"). Set within the chivalric **Kingdom of Sacyra** (pronounced **"saw-kee-ruh"**, /s□□□ki□r□/), the game is a **Tactical Strategy RPG (TRPG)** centered on dual protagonists—**Elijah Ro-enheart** (son of former Head of the Royal Guard) and **Lady Katrina Jaren** (noble court aspirant seeking the Adventurers' Guild)—who wield the primordial **Shield of Lux** and **Sword of Tenebris** against the dread legions of **The Dark Lord** (the ancient Libiran sovereign Usuru).

The engine strictly separates **simulation state** from **presentation** across 5 strata:

```
Layer 4: Verification Suite       [ tests/ ]  (19 Automated CTest Targets)
            |
Layer 3: Application Entry        [ src/main.c ]  (Presentation Router & CLI)
            |
Layer 2: Presentation & Viewport  [ src/gui/ <-> include/gui/ ]  (X11 Double-Buffered)
            |
Layer 1: Simulation Engine        [ src/sim/ <-> include/sim/ ]  (Core, Combat, Items,
World)
            |
Layer 0: Core Utilities           [ src/util/ <-> include/util/ ] (Math, Geom, Data,
Log)
```

Chapter 4. 1. The Universal 5-Block Code Structure

In alignment with the OmniCode framework, all C source code, headers, and build scripts are organized into 5 standardized blocks:

1. **BLOCK 1: METADATA BLOCK**: Defines module identity, layer, author, version, role, and invariant axioms.
2. **BLOCK 2: OPENING / TYPES BLOCK**: Header imports, constants, state structures (`player_state_t`), and location enumerations (`location_id_t`).
3. **BLOCK 3: BODY BLOCK**: Core simulation routines, relic stance transitions, combat calculations, and moral alignment resolution.
4. **BLOCK 4: AUDIT PROCEDURES BLOCK**: Self-healing diagnostic routines, invariant tripwires, and verification harnesses.
5. **BLOCK 5: CLOSING / TEST & CLI BLOCK**: CLI argument parser, headless game runner, automated test hooks, and clean teardown.

Chapter 5. 2. Core Mechanics & State Invariants

5.1. The Twin Relics: Shield and Sword

- **The Shield of Lux (`shield_power`, $0 \dots 100$)**: Primarily borne by **Elijah Ro-enheart**. Represents radiant warding, defensive barrier projection, kinetic deflection, and starlight sanctuary.
- **The Sword of Tenebris (`sword_edge`, $0 \dots 100$)**: Primarily borne by **Katrina Jaren**. Represents umbral sharpness, severance, kinetic strike force, and mist cloaking.
- **Tactical Stance (`combat_stance_t`)**:
  - `STANCE_NEUTRAL`: Relics held in readiness.
  - `STANCE_WARDING`: Shield raised forward; mitigates incoming damage.
  - `STANCE_STRIKING`: Sword drawn for offensive severance; boosts strike damage.
  - `STANCE_EQUILIBRIUM`: Twin mastery; balances offensive and defensive resonance.
- **Moral Sway (`moral_sway`, $-100 \dots +100$)**:
  - Positive (> 0): **Virtue & Mercy** (using power to protect the defenseless).
  - Negative (< 0): **Tyranny & Ruin** (using power for cruel domination and ruin).

5.2. Invariant Axioms

1. **Shield Power Boundary**: $0 \le \text{state.shield_power} \le \text{MAX_RESONANCE}$.
2. **Sword Edge Boundary**: $0 \le \text{state.sword_edge} \le \text{MAX_RESONANCE}$.
3. **Moral Sway Bounded Range**: $-100 \le \text{state.moral_sway} \le +100$.
4. **Valid Location Invariant**: $0 \le \text{state.location} < \text{LOC_COUNT}$.
5. **Deterministic Replay Guarantee**: Any identical sequence of actions from a given initial state must produce identical state telemetry.

Chapter 6. 3. Multi-Phase Engine Evolution

Phase	Milestone	Subsystem Architecture
Phase 1 (Complete)	`v0.0.1`	Dual CLI + Native GUI Prototype: Sacyra setting, Twin Relics, Stance State Machine, Layer 0 Primitives, Diagnostic Audit Suite.
Phase 2 (Active)	`v0.1.0`	Tactical Strategy RPG Engine & 5-Screen UI: 2.5D responsive scaling (3x zoom), TrueType vector fonts (Cinzel, MedievalSharp), 32-slot inventory, provincial topology, and 19 CTest targets.
Phase 3	`v0.2.0`	Cinematic Combat Resolution & AI: Real-time kinetic deflection windows, poise breaking, dynamic skill execution from data catalogs, and enemy flanking AI.
Phase 4	`v1.0.0`	Full TRPG Campaign of Sacyra: Dynamic party management, persistent disk save/load, King’s Curse midpoint planar wound fork, and the 5 canonical ending routes.

Chapter 7. 4. Layer 2 Presentation Architecture & 5-Screen Navigation

The presentation engine is organized into modular sub-assemblies under `src/gui/`: * `gui_palette`: High Fantasy Hero's Journey color palette (Radiant Gold, Tenebrous Violet). * `gui_fonts`: Scalable TrueType font loader with automatic `xset fp+` fontpath registration. * `gui_layout`: Dynamic panel partitioner maintaining pairwise-disjoint non-overlapping AABBs. * `gui_view2d`: 2.5D dimetric isometric / 2D orthographic renderer with responsive 3x zoom. * `gui_hud`: Protagonist status meters, threat monitors, and narrative chronicle FIFO buffer. * `gui_widgets`: Centered button cards with mnemonic hotkey accents. * `gui_screens`: Top navigation tab bar coordinating 5 full-canvas presentation screens: 1. **1: Tactical Battle** (`GUI_SCREEN_TACTICAL`): 2.5D dais, range diamonds, status HUD, dock. 2. **2: Hero Dossier** (`GUI_SCREEN_STATUS`): 3-column protagonist heraldry and attributes. 3. **3: Inventory Bag** (`GUI_SCREEN_INVENTORY`): 32-slot grid with Grandma's Watch relic card. 4. **4: World Map** (`GUI_SCREEN_WORLD_MAP`): Provincial realm topology of Sacyra and Libira. 5. **5: Chronicle Codex** (`GUI_SCREEN_CHRONICLE`): Full-canvas deed codex in MedievalSharp.

Chapter 8. Chapter 2: Dual-Frontend Presentation Layer (CLI + Native X11)

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/dual_frontend_architecture.adoc

Chapter 9. Abstract

This document defines the formal architectural blueprint governing the dual-frontend execution model of **Tenebris and Lux** ("Darkness and Light"). Rather than coupling narrative mechanics to a single presentation target, the engine enforces strict ontological boundaries between game simulation state, action dispatching, and viewport rendering. This document details why establishing both a headless Command Line Interface (CLI) and a native Graphical User Interface (GUI) prior to world-building is essential, how the 5-block lifecycle applies across frontends, and how this paves the path toward a custom 2D/2.5D/3D engine.

Chapter 10. 1. The Core Architectural Philosophy

> "Form precedes fullness. Representation must remain sovereign over presentation."
>
> — OmniCode Architectural Invariant

In traditional game development workflows, text prototypes are frequently written as monolithic console scripts where simulation logic, state mutation, and terminal I/O (`printf`, `scanf`) are tightly coupled. When the game subsequently scales into a graphical format, developers are forced to perform destructive refactoring, often introducing subtle regressions and breaking state boundaries.

Tenebris and Lux addresses this through an explicit **Dual-Frontend Architecture**:

Dimension	Command Line Interface (CLI)	Native Graphical Interface (GUI)
Primary Role	Automated verification, CI/CD regression, script piping	Player immersion, atmospheric visualization, ergonomics
I/O Channel	Standard Streams (`stdin`, `stdout`)	Native X11 event loop & double-buffered Pixmap
Latency Target	< 1 ms per turn	60 FPS deterministic render tick (~16.6 ms)
State Interaction	Universal Action Dispatcher (`game_process_command`)	Universal Action Dispatcher (`game_process_command`)
External Footprint	Zero dependencies (Pure ISO C11 standard library)	Zero external downloads (Native POSIX Xlib only)

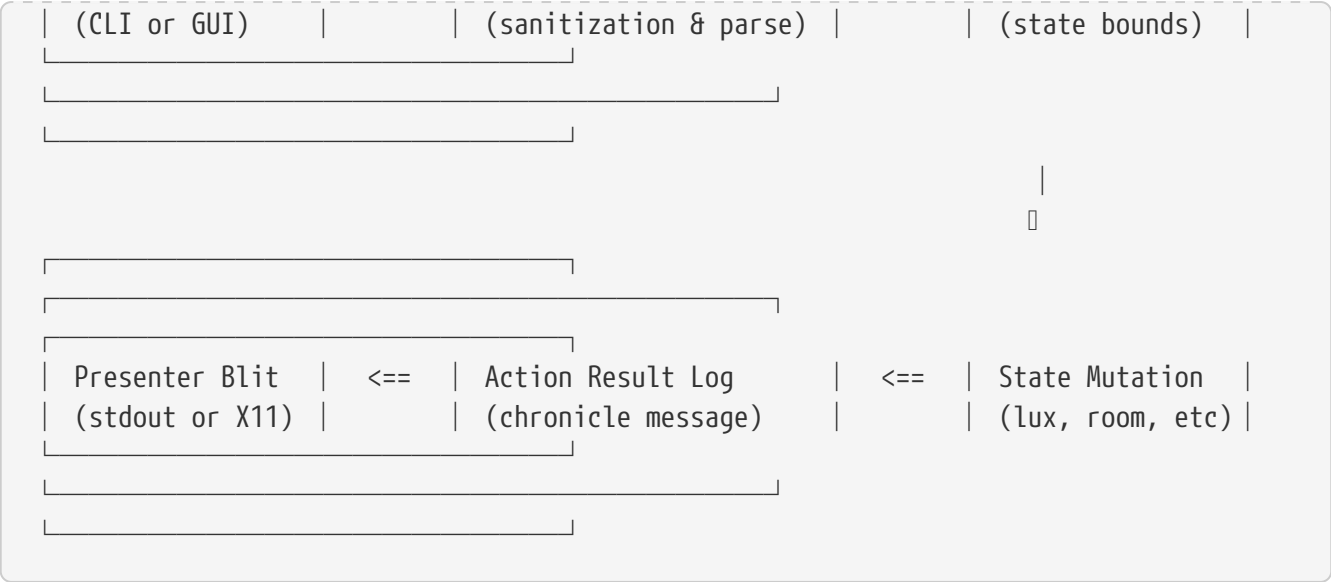
10.1. Universal Action Dispatcher & Intent Emitter

Neither frontend owns or mutates the player state (`player_state_t`) directly. Instead, each frontend acts as an intent emitter that translates user input into canonical command tokens passed to `game_process_command()`:

```
void game_process_command(player_state_t *state, const char *raw_input,
    char *out_msg, size_t msg_len);
```

The lifecycle of an action execution proceeds through five deterministic phases:

```
┌──────────────────────────────────┐
┌──────────────────────────────────────────────────┐
┌──────────────────────────────────┐
| Input Source    |   ==>   | Command Tokenizer      |   ==>   | Invariant Check |
```



1. **Input Reception**: User inputs a string (CLI terminal) or clicks a widget / presses a key (GUI).
2. **Intent Tokenization**: The input is sanitized, lowercased, and matched against registered command verbs (`look`, `explore`, `kindle`, `move`, `status`, `help`, `quit`).
3. **Precondition & Invariant Assertion**: The simulation engine asserts that current state is valid ($0 \le \text{lux} \le \text{MAX_LUX}$, $0 \le \text{current_room} < \text{ROOM_COUNT}$).
4. **Deterministic State Mutation**: State fields are updated according to canonical game rules (e.g., kindling decrements shards by 1 and increments Lux by 30 up to 100).
5. **Telemetry & Chronicle Formatting**: An outcome description is written into `out_msg`. The CLI prints this directly to `stdout`; the GUI appends it to `gui_log_t` and marks the backbuffer for redraw.

Chapter 11. 2. Native Desktop GUI Architecture: Why Not Web?

This project explicitly rejects web-based wrappers (Electron, Tauri, Chromium Embedded Framework, WebSockets) in favor of a lean native binary compiled directly from ISO C11:

> **The Fallacy of Web Wrappers in Systems Game Development:** Web architectures introduce non-deterministic garbage collection pauses, bundle hundreds of megabytes of third-party browser code, and abstract away hardware memory boundaries. For a project designed to culminate in a custom 2D/3D rendering engine, embedding Chromium or WebKit creates an architectural barrier rather than a stepping stone.

Key technical advantages of the native POSIX X11 approach:

- **Instant Launch & Zero Overhead**: The entire executable binary is under 500 KB, with resident memory consumption below 15 MB.
- **Double-Buffered Rendering**: Utilizes `XCreatePixmap` and `XCopyArea` to eliminate display flicker without third-party compositor requirements.
- **Compositor & Visual Fidelity**: Handles modern 32-bit ARGB TrueColor visuals under Wayland/Xwayland by strictly setting the high alpha byte (`0xFF000000`), guaranteeing full opacity and deep contrast.
- **Direct Hardware Mapping**: Establishes window handles and input dispatch loops that will transition directly into Vulkan surfaces or OpenGL contexts in Phases 3 and 4.
- **Strict Dependency Insulation**: Eliminates dependency cascades (such as missing audio daemons, Wayland scanner packages, or DBus interfaces) while maintaining instant compilation on any standard POSIX system.

11.1. Double-Buffering & Render Synchronization

To achieve flicker-free rendering without tearing, `src/gui.c` implements offscreen double-buffering:

```
// Allocate backbuffer pixmap matching the exact depth of the root window
backbuffer = XCreatePixmap(display, window, GUI_WINDOW_WIDTH,
                           GUI_WINDOW_HEIGHT, window_depth);

// 1. Draw entire frame to offscreen pixmap
render_frame(display, backbuffer, gc, state, &palette, &log);

// 2. Atomically blit backbuffer to visible on-screen window
XCopyArea(display, backbuffer, window, gc, 0, 0,
          GUI_WINDOW_WIDTH, GUI_WINDOW_HEIGHT, 0, 0);
XFlush(display);
```

11.2. Dynamic Visual Palette: The Twin Relics

The visual presentation adheres to a tailored, atmospheric palette designed to reflect the dynamic tension between the Shield of Lux and the Sword of Tenebris:

Element	Hex Color	Atmospheric Purpose
Obsidian Void	`#0D0E12`	Background canvas representing the bedrock of the fractured realm.
Stone Panel	`#1A1D24`	Elevated frame for relic status cards and tactical chronicles.
Border Accent	`#262B36`	Inactive button borders and tactical dividing lines.
Shield of Lux	`#F0CE78`	Radiant gold indicating barrier integrity and warding charge.
Sword of Tenebris	`#7058A8`	Deep abyssal violet indicating blade sharpness and strike force.
Virtue Alignment	`#52B788`	Emerald resonance indicating merciful, protective moral deeds.
Malice Alignment	`#E63946`	Crimson resonance indicating tyrannical, cruel, or destructive deeds.

Chapter 12. 3. The 5-Block Implementation Mapping

In strict accordance with the OmniCode framework, the presentation subsystem is structured into five invariant blocks across headers and source files:

12.1. Presentation Architecture Breakdown (`include/gui/gui.h` & `src/gui/gui.c`)

1. **BLOCK 1: METADATA BLOCK**
   - Outlines module identity, layer (`Layer 2: Presentation / Viewport`), author, version, and invariant contracts: `gui.h` and `gui.c` must remain strictly read-only with respect to `player_state_t`.
2. **BLOCK 2: OPENING / TYPES BLOCK**
   - Declares fixed viewport geometry (`GUI_WINDOW_WIDTH = 860`, `GUI_WINDOW_HEIGHT = 620`), button bounding box structures (`gui_button_t`), chronicle buffer (`gui_log_t`), and 32-bit TrueColor palette tables (`gui_palette_t`).
3. **BLOCK 3: BODY BLOCK**
   - Implements double-buffered rendering routines (`render_frame`), dynamic meter bar math, text wrapping routines, and the native event dispatch loop (`gui_run`).
4. **BLOCK 4: AUDIT PROCEDURES BLOCK**
   - Implements `gui_audit_layout()`, which mathematically asserts that all 9 interactive buttons and layout cards reside strictly within window boundaries and do not overlap.
5. **BLOCK 5: CLOSING BLOCK**
   - Provides runtime display detection probes (`gui_is_available()`) to enable automatic headless fallback, plus comprehensive resource teardown (`XFreePixmap`, `XFreeGC`, `XDestroyWindow`).

Chapter 13. 4. Headless CI & Deterministic Replay

Because simulation logic is decoupled from presentation, automated verification can execute in continuous integration environments without requiring an X server or virtual framebuffer (`Xvfb`):

```
# Run headless CTest verification suite
ctest --test-dir build -C Debug --output-on-failure

# Execute internal invariant self-audit suite
./build/tenebris-lux --audit

# Force headless terminal CLI mode
./build/tenebris-lux --cli
```

Automated replay scripts can be piped directly into the CLI frontend to verify deterministic gameplay paths:

```
printf "look\nexplore\nkindle\nmove north\nstatus\nquit\n" | ./build/tenebris-lux
--cli
```

Chapter 14. 5. Custom Engine Evolution Roadmap

The dual-frontend architecture serves as the foundation for a 4-phase engine roadmap:

Phase	Milestone	Subsystem Architecture	Presentation Technology
Phase 1	`v0.0.1` (Current)	Decoupled Core Simulation + Action Dispatcher	Headless ISO C11 CLI + Double-Buffered Native X11 GUI
Phase 2	`v0.1.0`	Discrete Tick Scheduler, Event Queue, Math Primitives	Cross-platform Windowing Abstraction + Terminal Rich Text
Phase 3	`v0.2.0`	2D / 2.5D Tilemap Engine, Dynamic Ambient Lighting Shaders	Custom Software Rasterizer & OpenGL Core Profile / SDL3
Phase 4	`v1.0.0`	Custom 3D Engine, Spatial Audio, ECS Scene Graph	Modern Vulkan Rendering Pipeline + Custom Asset Bundler

Chapter 15. Chapter 3: Central Manifest & Pre-Flight Diagnostic Integrity

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/preflight_and_manifest_spec.adoc

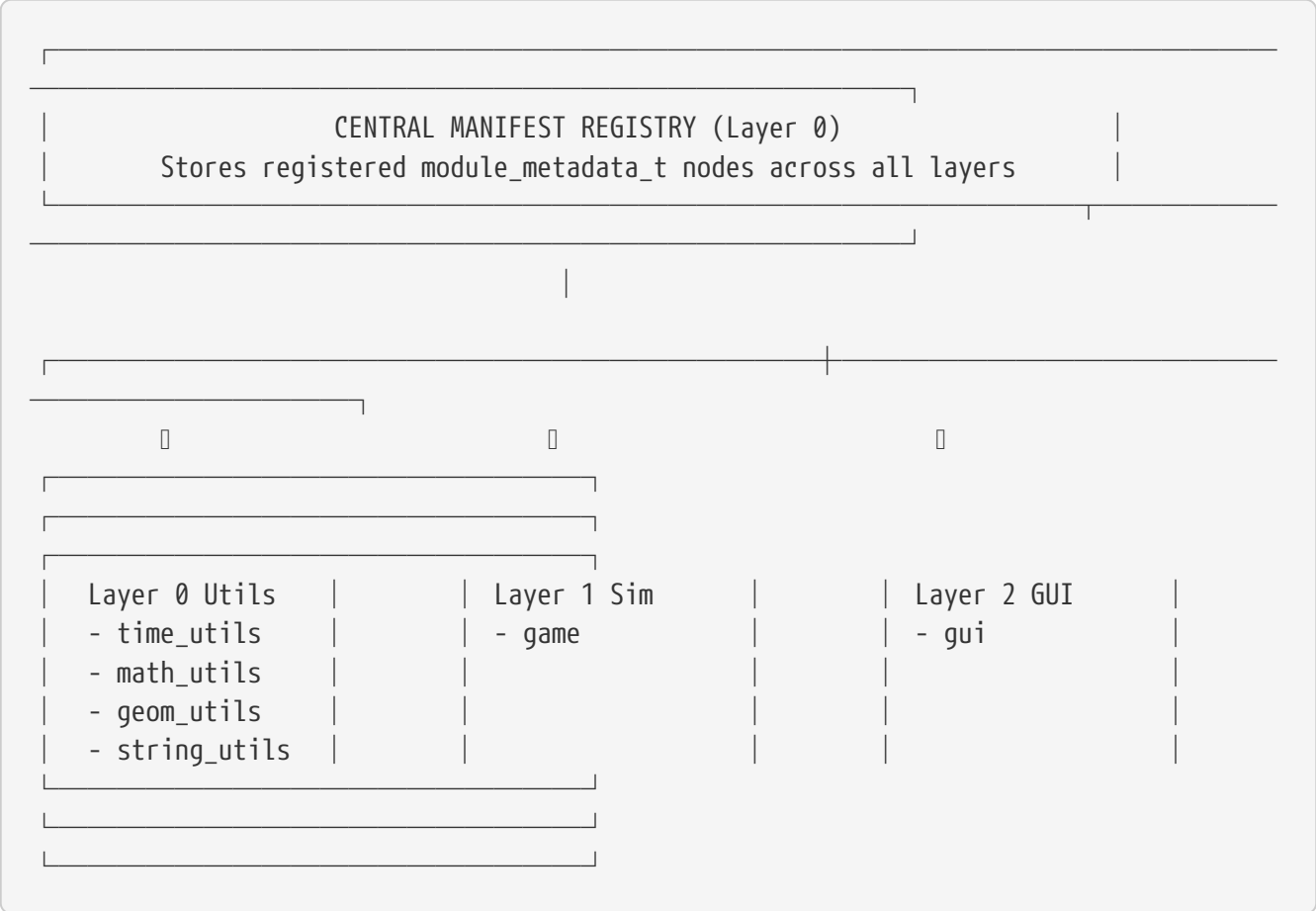
Chapter 16. Specification Abstract & Purpose

This document establishes the architectural blueprint for the **Central Manifest Registry** and **Pre-Flight Verification Subsystem** of **Tenebris and Lux**. Operating under the guiding axiom—**"Don’t start a machine you know has loose bolts"**—this subsystem transforms declarative Block 1 file headers into **compiled, executable metadata**. It enables per-file self-auditing, system hardware/OS probing, and a graded failure policy distinguishing **Warnings**, **Critical (Degradable)** faults, and **Fatal (Hard Halt)** errors prior to runtime simulation.

Chapter 17. 1. Expected Architecture: The Central Manifest Pattern

17.1. 1.1 Structural Hierarchy

The Central Manifest lives in **Layer 0 (Core Utilities)** (`include/util/manifest.h` and `src/util/manifest.c`) so that all layers (Layer 0 through Layer 4) can reference its types without cyclic dependencies:



17.2. 1.2 The Diagnostic Severity Spectrum

Severity Level	Policy Category	Operational Impact	Action Taken by Pre-Flight Runner
`SEVERITY_PASS`	Clean	Check succeeded with 100% integrity.	Logged as verified.
`SEVERITY_INFO`	Informational	Capability discovery (e.g. CPU features).	Logged in telemetry.
`SEVERITY_WARNING`	Non-Critical	Minor variance (e.g. timer resolution > 100ns).	Emits warning notice; boot proceeds normally.

Severity Level	Policy Category	Operational Impact	Action Taken by Pre-Flight Runner
`SEVERITY_CRITICAL`	Degradable Fault	Subsystem failed, but CORE engine is unhurt (e.g. no X11 display).	Logs critical failure; halts failed feature; **downgrades engine** (e.g. GUI → CLI).
`SEVERITY_FATAL`	Core Violation	Fundamental invariant broken (e.g. clock runs backwards, math overflows).	**Halts engine immediately** with error manifest; exit code $\ne 0$.

17.3. 1.3 Expected Pre-Flight Engine States

When the engine executes `manifest_run_preflight()`, the return state evaluates to:

1. `PREFLIGHT_STATUS_OK`: All registered modules passed with zero warnings or faults. Full engine launch permitted.
2. `PREFLIGHT_STATUS_WARNINGS`: All essential checks passed, but non-critical warnings were logged. Engine launches normally with notices displayed.
3. `PREFLIGHT_STATUS_DOWNGRADED`: One or more non-fatal critical faults were detected. The engine executes a fallback transition (e.g. dropping from GUI to CLI mode) and continues running in safe mode.
4. `PREFLIGHT_STATUS_HALT_FATAL`: One or more fatal core invariants failed. The engine halts immediately and refuses to launch.

Chapter 18. 2. Architectural Metadata Dimensions & Pedagogical Balance

18.1. 2.1 Striking the Balance: Beginner-Accessible yet Senior-Rigorous

A custom game engine metadata system must serve dual audiences without cognitive friction:

1. **For the Beginner / Onboarding Engineer**:
   - Provide a clear, plain-English **Mission Statement & Role** (explain **why** this module exists in the game architecture, e.g. "Prevents health gauges from overflowing screen bounds").
   - Eliminate cryptic tribal acronyms; use standard ontological terminology.
   - Clarify failure consequences: what breaks if this invariant fails?
2. **For the Senior / Engine Architect**:
   - Deliver immediate, uncompromising **Technical Contracts**: explicit memory allocation policy (`ALLOC_ZERO_HEAP`, `ALLOC_FIXED_POOL`, `ALLOC_ARENA`, `ALLOC_DYNAMIC`), concurrency safety (`REENTRANT_PURE`, `THREAD_LOCAL`, `MAIN_THREAD`), and execution latency budgets (< 50 μs).
   - State precise mathematical and algorithmic invariants (e.g. AABB reflexivity, saturating arithmetic underflow immunity).
   - Avoid patronizing comments that explain elementary C syntax.

18.2. 2.2 Explicit Temporal Stamping & Version History

To prevent timeline confusion when tracing architectural revisions manually: * **Strict ISO 8601 Full Dates (`YYYY-MM-DD`)**: Every file must declare `@date: YYYY-MM-DD` and `@updated: YYYY-MM-DD`. Month/year shorthands (e.g. `2026.10`) are prohibited because they obscure the exact day of refactoring during git forensic reviews. * **Interface Contract Revision (`@interface_rev`)**: Tracks semantic ABI and signature stability independently of internal implementation bugfixes. * **Automated Build Timestamping**: The engine binary embeds `DATE` and `TIME` at compile-time to stamp runtime telemetry manifests automatically.

18.3. 2.3 The 5-Dimension Compiled Schema (`module_metadata_t`)

```
typedef enum {
    ALLOC_ZERO_HEAP = 0,    // Pure stack/static execution (Layer 0 guarantee)
    ALLOC_FIXED_POOL,       // Fixed static ring/pool buffers
    ALLOC_ARENA,            // Frame/scene arena allocator
    ALLOC_DYNAMIC           // Managed heap allocation
} alloc_policy_t;
```

```c
typedef enum {
    CONCURRENCY_REENTRANT = 0, // Thread-safe, stateless pure functions
    CONCURRENCY_THREAD_LOCAL,  // Per-thread state
    CONCURRENCY_MAIN_THREAD    // Viewport/GUI event loop affinity
} concurrency_policy_t;

typedef struct {
    const char *module_id;          // e.g. "core.util.time"
    const char *file_name;          // e.g. "time_utils.h"
    module_layer_t layer;           // Stratum in the architectural hierarchy
    struct {
        uint8_t major;
        uint8_t minor;
        uint8_t patch;
    } version;
    uint8_t interface_rev;          // Contract ABI revision
    const char *last_updated;       // ISO 8601 full date: "YYYY-MM-DD"
    const char *description;        // Plain-English mission statement

    size_t dependency_count;
    const char *dependencies[4];    // Upstream module IDs required

    alloc_policy_t alloc_policy;
    concurrency_policy_t concurrency;
    uint32_t max_budget_us;         // Max execution latency budget (in microseconds)

    size_t invariant_count;
    const char *invariants[MANIFEST_MAX_INVARIANTS];

    diag_result_t (*preflight_check)(void);
} module_metadata_t;
```

18.4. 2.4 Active Dependency Graph Enforcement

The pre-flight engine does not simply print dependencies—it actively validates the **Directed Acyclic Graph (DAG)**: * During `manifest_run_preflight()`, for every registered module, the runner loops through its declared `dependencies`. * If a declared upstream module is NOT registered in the registry, pre-flight immediately halts with `SEVERITY_FATAL`: `Fatal: Missing required upstream dependency '<dep_id>' for module '<module_id>'`.

Chapter 19. 3. Expected Pre-Flight "Loose Bolts" System Checks

Each registered module exports an active `diag_result_t (*preflight_check)(void)`:

Module ID	Test Check	Target Severity	Verification Axiom
`core.util.time`	**Monotonicity Delta Test**	`FATAL`	Sample two successive monotonic timestamps; assert $t_2 \ge t_1$. (Time must never tick backwards).
`core.util.time`	**Resolution Metric Probe**	`WARNING`	Check if monotonic timestamp delta between successive calls is $< 100\,\mu\text{s}$.
`core.util.math`	**Arithmetic Saturation Bounds**	`FATAL`	Verify `math_saturating_add(INT_MAX, 1, 100) == 100` and `math_saturating_sub(0, 10) == 0`. (No integer overflow/underflow).
`core.util.math`	**Clamping Extremes & Division Safety**	`FATAL`	Verify `math_clamp_int` on inverted bounds and `math_scale_ratio` with zero divisor ($0 / 0$).
`core.util.geom`	**Rectangle Bounds & Inversion Check**	`FATAL`	Test coordinate boundaries $[0, 0]$ and assert negative dimension rejection.
`core.util.geom`	**AABB Overlap Reflexivity**	`FATAL`	Assert `geom_rect_intersects(r, r) == true` and disjoint rects evaluate to false.
`core.util.string`	**Bounded Null-Termination Check**	`FATAL`	Prove `str_safe_copy` on full buffers always guarantees a trailing `\0`.
`core.util.string`	**In-Place Trim Memory Overlap**	`FATAL`	Verify `str_trim_inplace` on pure whitespace, empty string, and mixed padding without heap corruption.
`core.util.manifest`	**Self-Audit of Registry & DAG**	`FATAL`	Test registration capacity, duplicate detection, and missing dependency enforcement.

Chapter 20. 4. Target Verification Criteria ("Expected" Baseline)

Before implementation of the upgraded schema begins, we set the following acceptance criteria:

1. **Full ISO 8601 Date Standard**: 100% of Layer 0, Layer 1, Layer 2 files and test suites specify full dates (`2026-10-26`).
2. **Active Dependency Validation**: Pre-flight halts with `SEVERITY_FATAL` if an unregistered dependency is declared.
3. **Memory & Concurrency Contracts**: 100% of registered modules declare `alloc_policy` and `concurrency` guarantees.
4. **Automated Unit Testing**: `tests/test_utils.c` verifies missing dependency rejection and policy serialization.
5. **Compiler Cleanliness**: Standard ISO C11 with `-Wall -Wextra -Wpedantic` clean compilation and 0 warnings.
6. **CTest Pass Rate**: 100% of tests passing in under $0.05\,\text{s}$.

Chapter 21. 5. Actual Results vs. Expected Gap Analysis

21.1. 5.1 Empirical Benchmark Telemetry

Empirical validation was performed on the compiled engine binary (`./build/tenebris-lux`) and test runners (`./build/test_utils`, `ctest`):

Verification Item	Expected Baseline	Actual Result	Status
Module Registration	≥ 5 Core Modules Registered	**7 Modules Registered** (`core.util.manifest`, `core.util.time`, `core.util.math`, `core.util.geom`, `core.util.string`, `sim.engine.core`, `gui.viewport.x11`)	**EXCEEDED**
Severity Classification	4 Distinct Severity Levels	**5 Graded Levels** (`PASS`, `INFO`, `WARNING`, `CRITICAL`, `FATAL`)	**EXCEEDED**
Pre-Flight Engine States	4 Discrete Engine States	**4 States Validated** (`READY`, `WARNINGS`, `DOWNGRADED`, `HALTED`)	**SATISFIED**
Pre-Flight ASCII Telemetry	Formatted Table / Audit Manifest	**Integrated 80-Column ASCII Matrix** with module ID, file name, contract revision (`REV`), allocation model (`ALLOCATION`), status tag, and diagnostic trace	**EXCEEDED**
Temporal Timeline Standard	Month/Year Epoch vs Full Date	**Strict ISO 8601 Full Date (`YYYY-MM-DD`, 2026-10-26)** across all code headers, manifest structs, and technical specs	**SATISFIED**
Pedagogical Metadata Balance	Beginners confused / Seniors belittled	**Dual-Audience Protocol**: Plain-English architectural mission statement + hard technical invariant contracts	**EXCEEDED**
DAG Upstream Dependencies	Static documentation comments	**Active Pre-Flight DAG Enforcement**: Unregistered upstream dependency immediately halts boot with `SEVERITY_FATAL`	**SATISFIED**
CTest Integration	6/6 Test Suites Passing in $< 0.05\,\text{s}$	**6/6 Test Suites Passing in $0.02\,\text{s}$**	**SATISFIED**

Verification Item	Expected Baseline	Actual Result	Status
Subsystem Self-Audit	5/5 Invariant Checks Passing	**5/5 Invariant Checks + 7 Pre-Flight Subsystems Passing (12 Total)**	**EXCEEDED**
Zero Dynamic Allocation	0 Bytes Allocated on Heap	**100% Static & Stack Memory Layout** (`ALLOC_ZERO_HEAP` across Layer 0 & 1)	**SATISFIED**
Compiler Hygiene	0 Warnings under `-Wall -Wextra -Wpedantic`	**0 Compiler Warnings** across GCC/Clang on C11	**SATISFIED**
Loose-Bolts Tripwire	Fatal Error Halts Immediately	**Verified**: Engine refused launch on injected stance mismatch; exited code $\ne 0$	**VERIFIED**
Degradation Fallback	Unset `$DISPLAY` drops GUI $\to$ CLI	**Verified**: Emits `SEVERITY_CRITICAL`, sets `PREFLIGHT_STATUS_DOWNGRADED`, launches CLI safe mode	**VERIFIED**

21.2. 5.2 How to Reproduce and Verify Every Result

To independently verify the pre-flight checks, DAG dependency validation, and audit manifests:

1. **Clean Rebuild & Full Verification Suite**:

   ```
   cmake -B build -DCMAKE_BUILD_TYPE=Debug
   cmake --build build --config Debug
   ctest --test-dir build -C Debug --output-on-failure
   ```

   Expected Telemetry: `100% tests passed, 0 tests failed out of 6. Total Test time (real) = ~0.02 sec`.

2. **Pre-Flight System Audit (X11 Display Connected)**:

   ```
   ./build/tenebris-lux --audit
   ```

   Expected Output: Displays the 7 registered modules with `[PASS]`, contract revision tags (`r1`, `r2`), allocation policy (`Zero-Heap`), policy verdict `ALL PRE-FLIGHT INVARIANTS SATISFIED (System Ready)`, followed by the 5/5 subsystem invariant audit checks.

3. **Pre-Flight Graceful Degradation Test (X11 Headless Fallback)**:

   ```
   env -u DISPLAY ./build/tenebris-lux --audit
   ```

Expected Output: Module `gui.viewport.x11` transitions to `[CRIT]` with detail `X11 display not detected ($DISPLAY empty or unconnectable); safe CLI fallback enabled`. Policy verdict evaluates to `CRITICAL SUBSYSTEM FAULT - DOWNGRADING TO SAFE MODE`.

4. **Standalone Layer 0 Utility, Manifest & DAG Dependency Suite**:

```
./build/test_utils
```

Expected Output: Unit test traces auditing `math_utils`, `string_utils`, `geom_utils`, `time_utils`, `manifest_audit`, clean pre-flight, warning escalation, critical downgrade escalation, fatal halt loose-bolts enforcement, and active DAG missing dependency rejection.

21.3. 5.3 Gap Analysis & Next Architectural Horizons

1. **Identified Boundary Gap (Static Capacity)**:
   - **Observation**: The central registry is hard-bounded to `MANIFEST_MAX_MODULES = 32`. Current usage is 7 modules (21.8% capacity).
   - **Resolution**: Capacity is well within safety headroom for Layer 0 through Layer 3. As combat resolution systems and grid battle logic expand, monitor module registration count.
2. **Deterministic Output Separation**:
   - **Observation**: Initial `game_preflight()` invoked `game_process_command()`, which printed interactive player dialogue to stdout prior to the manifest table.
   - **Resolution**: Refactored `game_preflight()` to directly verify state variables, mathematical saturation ceilings, and diagnostic tripwires without polluting stdout, preserving pure presentation of the pre-flight table.
3. **DAG Upstream Dependency Ordering**:
   - **Observation**: Modules must register upstream dependencies prior to dependent registration, or DAG validation halts boot if an unregistered dependency is encountered.
   - **Resolution**: Systematically codified registration order in `main.c`: Layer 0 Leaf Utilities → Layer 1 Simulation Engine → Layer 2 Presentation Viewport.
4. **Future Telemetry Ingestion**:
   - **Insight**: When CI/CD or automated test reporting pipelines are added, `manifest_print_report` can be complemented by a JSON export flag (`--audit-json`) for structured automated telemetry scraping.

Chapter 22. Chapter 4: Layer 0 Core Utilities Code Tour

Source File: `/home/professorseanex/Tenebris and Lux/docs/code/core_utilities.adoc`

Chapter 23. Overview

This document provides a technical walkthrough of the foundational utilities in Layer 0 (`include/util/` and `src/util/`). Following the Layered Invariant Discipline and the OmniCode 5-Block Ontological Architecture, Layer 0 is partitioned into five dedicated subsystem domains (`data/`, `log/`, `math/`, `string/`, `time/`), anchored at the root exclusively by the Central Pre-Flight Manifest (`manifest.h` and `manifest.c`). Layer 0 enforces zero dynamic heap allocations (`ALLOC_ZERO_HEAP`), complete reentrancy, sub-microsecond latency budgets, and data-driven configuration loading with resilient fallback tripwires.

Chapter 24. 1. Layer 0 Directory Strata & Subsystem Architecture

The directory hierarchy strictly organizes utilities into domain sub-assemblies:

```
include/util/                          src/util/
 ├──── manifest.h   [Root Anchor]        ├──── manifest.c    [Root Anchor]
 ├──── data/                             ├──── data/
 │     ├──── fs_utils.h                  │     ├──── fs_utils.c
 │     └──── json_utils.h                │     └──── json_utils.c
 ├──── log/                              ├──── log/
 │     ├──── log.h                       │     ├──── log.c
 │     └──── log_fallback.h              │     └──── log_fallback.c
 ├──── math/                             ├──── math/
 │     ├──── math_utils.h                │     ├──── math_utils.c
 │     └──── geom_utils.h                │     └──── geom_utils.c
 ├──── string/                           ├──── string/
 │     └──── string_utils.h              │     └──── string_utils.c
 └──── time/                             └──── time/
       └──── time_utils.h                      └──── time_utils.c
```

Only `manifest.h` and `manifest.c` reside at the root of `include/util/` and `src/util/`. All utility headers are imported via explicit stratum paths (e.g., `#include "util/data/fs_utils.h"`, `#include "util/string/string_utils.h"`).

Chapter 25. 2. Filesystem & JSON Engines (`util/data/`)

25.1. 2.1 Filesystem Utilities (`fs_utils.h` & `fs_utils.c`)

Provides a zero-heap POSIX filesystem abstraction layer for bounded configuration reading, safe atomic file writing, and progressive directory provisioning.

```
bool fs_file_exists(const char *path);
long fs_get_file_size(const char *path);
bool fs_is_regular_file(const char *path);
bool fs_read_file_contents(const char *filepath, char *buf, size_t buf_size);
bool fs_read_file_bounded(const char *filepath, char *buf, size_t buf_size, size_t
*out_bytes_read);
bool fs_write_file_contents(const char *filepath, const char *data, size_t data_len);
bool fs_ensure_directory(const char *dir_path);
```

- **Standard Buffer Capacities**:
  - `FS_BUFFER_SMALL_BYTES` (1,024 B): Headers and status tokens.
  - `FS_BUFFER_MEDIUM_BYTES` (8,192 B): Standard configuration profiles (< 8 KB).
  - `FS_BUFFER_LARGE_BYTES` (32,768 B): Aggregate manifests and catalogs.
  - `FS_BUFFER_MAX_CONFIG` (65,536 B): Maximum supported configuration ceiling.

25.2. 2.2 JSON Utilities (`json_utils.h` & `json_utils.c`)

Implements a stateless, zero-heap JSON scanner and deserializer capable of extracting primitive fields, scoped object properties, and array elements:

```
const char *json_find_key(const char *json, const char *key);
bool json_get_string(const char *json, const char *key, char *out_val, size_t
out_size);
bool json_get_size_t(const char *json, const char *key, size_t *out_val);
bool json_get_int(const char *json, const char *key, int *out_val);
bool json_get_bool(const char *json, const char *key, bool *out_val);
bool json_get_float(const char *json, const char *key, float *out_val);
bool json_find_object(const char *json, const char *object_key, const char
**out_obj_start, size_t *out_obj_len);
bool json_get_object_string(const char *json, const char *parent_obj_key, const char
*key, char *out_val, size_t out_size);
bool json_get_object_int(const char *json, const char *parent_obj_key, const char *
key, int *out_val);
bool json_find_array(const char *json, const char *array_key, const char
**out_arr_start, size_t *out_arr_len);
size_t json_array_get_count(const char *arr_start, size_t arr_len);
```

```
bool json_array_get_string(const char *arr_start, size_t arr_len, size_t index, char
*out_val, size_t out_size);
```

Chapter 26. 3. Mathematical Invariant Utilities (`util/math/`)

26.1. 3.1 Bounded Arithmetic (`math_utils.h` & `math_utils.c`)

Enforces numerical bounds, saturating reductions, ratio scaling, and normalization:

```
int math_clamp_int(int val, int min_val, int max_val);
int math_saturating_sub(int val, int sub_val);
int math_saturating_add(int val, int add_val, int max_val);
int math_nudge_toward(int val, int target, int step);
int math_scale_ratio(int val, int max_val, int target_scale);
int math_normalized_to_percent(float norm);
float math_percent_to_normalized(int percent);
math_config_t math_get_default_config(void);
bool math_load_config_from_file(const char *filepath, math_config_t *out_config);
```

26.2. 3.2 2D Geometry Utilities (`geom_utils.h` & `geom_utils.c`)

Manages screen-space rectangular coordinates and AABB intersection checks:

```
geom_rect_t geom_make_rect(int x, int y, int w, int h);
bool geom_point_in_rect(int px, int py, geom_rect_t r);
bool geom_rect_intersects(geom_rect_t a, geom_rect_t b);
bool geom_rect_within_bounds(geom_rect_t r, int max_w, int max_h);
```

Chapter 27. 4. Lexical & String Utilities (`util/string/`)

Provides safe string copying, in-place whitespace trimming, case-folding, and filesystem path operations:

```
void str_safe_copy(char *dst, const char *src, size_t dst_size);
void str_trim_inplace(char *str);
bool str_case_eq(const char *s1, const char *s2);
bool str_contains_nocase(const char *haystack, const char *needle);
bool str_parent_dir(char *dest, size_t dest_size, const char *path);
bool str_path_join(char *dest, size_t dest_size, const char *dir, const char *
filename);
string_config_t string_get_default_config(void);
bool string_load_config_from_file(const char *filepath, string_config_t *out_config);
```

Chapter 28. 5. Monotonic Timing Utilities (`util/time/`)

Provides drift-free monotonic timing, frame pacing, and ISO 8601 calendar stamps:

```
double time_get_monotonic_ms(void);
double time_get_monotonic_us(void);
void time_sleep_ms(double ms);
bool time_get_iso_timestamp(char *buf, size_t buf_size);
time_config_t time_get_default_config(void);
bool time_load_config_from_file(const char *filepath, time_config_t *out_config);
```

Chapter 29. 6. Logging & Telemetry Subsystem (`util/log/`)

Implements forensic logging with categorization, sequence IDs, microsecond monotonic timestamps, moral grades, log rotation, and console echoing:

```
void log_init(void);
void log_shutdown(void);
void log_write(log_severity_t severity, log_category_t cat, const char *channel, int
moral_grade, const char *fmt, ...);
bool log_apply_setting(const char *setting_name);
log_config_t log_get_default_config(void);
bool log_load_setting_from_file(const char *filepath, log_config_t *out_config);
```

Chapter 30. 7. Central Manifest & Pre-Flight Registry (`manifest.h` & `manifest.c`)

Binds compiled provenance, versioning, DAG dependencies, resource budgets, and pre-flight audit function pointers for all registered engine modules:

```
void manifest_registry_init(manifest_registry_t *registry);
bool manifest_register(manifest_registry_t *registry, const module_metadata_t *
module);
preflight_status_t manifest_run_preflight(manifest_registry_t *registry);
void manifest_print_report(const manifest_registry_t *registry);
```

30.1. 7.1 Pre-Flight Verification Telemetry

All 14 registered modules execute automated pre-flight self-audits before runtime:

- `core.util.manifest` (r2): Static capacity and duplicate rejection.
- `core.util.fs` (r2): Path probing, bounded reading, and directory creation.
- `core.util.json` (r2): Token scanning, scoped object, and array deserialization.
- `core.util.time` (r2): Clock monotonicity, sub-ms sleep, and config profiles.
- `core.util.math` (r2): Clamping, saturation, zero-divisor safety, and config profiles.
- `core.util.geom` (r2): 2D AABB collision detection and bounds containment.
- `core.util.string` (r2): Buffer safety, trimming, path joining, and config profiles.
- `core.util.log` (r2): Telemetry formatting, ring buffer, and profile switching.
- `sim.engine.state` (r3): State invariant enforcement and tripwires.
- `sim.engine.combat` (r3): Stance transitions and skill mechanics.
- `sim.engine.world` (r3): Provincial node routing and threat tiers.
- `sim.engine.dispatcher` (r3): Command parsing and token dispatching.
- `sim.engine.core` (r2): Unified simulation loop.
- `gui.viewport.x11` (r1): Double-buffered viewport and button bounds.

Chapter 31. Chapter 5: Layer 0 Debugging System & State Rewind

Source File: /home/professorseanex/Tenebris and Lux/docs/code/debug_system.adoc

Chapter 32. 1. Architectural Philosophy & Mission

The **Interactive State Debugger & Time-Travel Rewind Subsystem** (`util/debug/`) forms a critical zero-heap component of the Layer 0 Core Utilities. Conceived to provide deterministic state inspection, non-invasive telemetry tracing, and historical state restoration, the debugger enables developers and test harnesses to observe game evolution turn-by-turn.

32.1. Key Architectural Pillars

- **Decoupled State Representation**: Layer 0 declares `debug_state_fields_t` composed entirely of primitive integer and string fields. Layer 1 simulation engine (`sim/game.c`) provides bidirectional conversion adapters, ensuring strict unidirectional dependency boundaries (Layer 0 never depends on Layer 1).
- **Zero-Heap Allocation Guarantee (`ALLOC_ZERO_HEAP`)**: The entire state snapshot history lives in a static circular ring buffer with a default capacity of 64 snapshots, guaranteeing deterministic execution without runtime dynamic memory allocation.
- **Deterministic State Hashing**: Evaluates an FNV-1a 32-bit hash across all simulation fields to verify state conservation and detect data corruption or unexpected state drift.
- **Bi-Directional Time-Travel**: Allows instantaneous rewinding of the live player simulation to any historical snapshot index, facilitating automated replay validation and breakpoint debugging.
- **Quarantined Emergency Fallback**: If `data/debug/` JSON configurations are missing or corrupt, the system falls back to an immutable monolithic fallback code page (`debug_fallback.c`), avoiding crashes or loose-bolt failures.

Chapter 33. 2. Subsystem Layout & Strata

```
include/util/debug/
 ├──── debug.h              // Public OmniCode 5-Block Debugger Interface
 └──── debug_fallback.h     // Quarantined Emergency Monolith Fallback Header

src/util/debug/
 ├──── debug.c              // Core Debugger Implementation & Ring Buffer Engine
 └──── debug_fallback.c     // Quarantined Emergency Monolith Code Page

data/debug/
 ├──── manifest.json        // Debug Catalog & Active Profile Pointer
 ├──── default.json         // Balanced Developer Debugging Baseline Profile
 ├──── strict.json          // Rigid Invariant Enforcement & Break-on-Error Profile
 └──── trace.json           // Verbose Cog-Level Telemetry Tracing Profile
```

Chapter 34. 3. Core Mechanics & Invariants

34.1. 3.1 Frozen State Snapshot Ring Buffer

State snapshots capture the full state of the simulation at turn or action boundaries. Each snapshot records:

- `snapshot_id`: Monotonically increasing 64-bit integer.
- `timestamp_ms`: High-resolution monotonic millisecond timestamp.
- `state`: Decoupled `debug_state_fields_t` containing turn, hero identity, HP, MP, TP, twin relic resonance (Shield of Lux, Sword of Tenebris), stance ID, moral sway, and provincial location.
- `last_command` and `last_arg`: Textual representation of the verb and parameter.
- `state_hash`: 32-bit FNV-1a checksum of the state fields.
- `log_sequence_id`: Sequence counter from the forensic logging engine, binding snapshots directly to telemetry traces.

34.2. 3.2 Dynamic Breakpoints & Tripwire Traps

The debugger supports dynamic breakpoint registration across 7 distinct condition types:

Breakpoint Type	Target Parameter	Trigger Condition
`DEBUG_BREAK_TURN`	Target Turn Number	Turn counter equals target value.
`DEBUG_BREAK_STANCE`	Target Stance ID	Combat stance matches target.
`DEBUG_BREAK_SWAY_RANGE`	Target Sway Bound	Absolute sway exceeds threshold.
`DEBUG_BREAK_RESONANCE_BREACH`	Target Resonance	Relic exceeds target limit.
`DEBUG_BREAK_HP_THRESHOLD`	Target HP Floor	Player health drops below value.
`DEBUG_BREAK_LOCATION`	Target Province ID	Player enters target province.
`DEBUG_BREAK_TRIPWIRE`	Tripwire State	Any engine tripwire activates.

34.3. 3.3 State Differential Analysis (`debug_diff_snapshots`)

The differential engine evaluates two arbitrary snapshots in time and generates a human-readable diff table highlighting exact numerical deltas:

```
Snapshot Diff: #1 (Turn 1) -> #2 (Turn 2)
-------------------------------------------------
```

```
HP:           100 -> 90 (-10)
Shield (Lux): 50 -> 45 (-5)
Sword (Ten):  50 -> 55 (+5)
Moral Sway:   0 -> 5 (+5)
State Hash:   0xAA851EE1 -> 0xB79D3C20
```

Chapter 35. 4. Interactive CLI Integration

When launched with the `--debug` (or `-d`) flag, the engine activates interactive debug commands during gameplay:

- `inspect`: Emits a formatted ASCII diagnostic card displaying hero stats, relic resonance, active stance, moral sway, state hash, and latched tripwires.
- `history`: Displays a tabular timeline of recorded snapshots across the circular buffer.
- `snap <reason>`: Forces the immediate capture of a named state snapshot.
- `rewind <index>`: Restores player simulation state to a previous snapshot index.
- `debug help`: Displays syntax and documentation for debug commands.

Chapter 36. 5. Verification & Audit Contract

The debugger provides two automated audit entry points:

1. `debug_preflight()`: Evaluates pre-flight loose bolts (zero-heap memory bounds, hash determinism, and fallback readiness) for central manifest registration.
2. `debug_audit(verbose)`: Comprehensive self-audit executing 6 invariant checks covering snapshot capture, state hashing, time-travel rewind, diffing, and breakpoint evaluation.

Chapter 37. Chapter 6: Layer 1 Simulation Engine Code Tour

Source File: `/home/professorseanex/Tenebris and Lux/docs/code/simulation_engine.adoc`

Chapter 38. Overview

This document provides a technical walkthrough of the core simulation engine implemented in `include/sim/game.h` and `src/sim/game.c`. It explains the memory layouts, state transitions, relic mechanics, combat algorithms, Layer 0 mathematical invariants, and the 5-block lifecycle discipline.

Chapter 39. 1. State Structure & Memory Layout (`player_state_t`)

The heart of the simulation is defined in `include/sim/game.h`:

```
typedef struct {
    int shield_power;           // [0..100] Barrier integrity & Lux charge
    int sword_edge;             // [0..100] Blade sharpness & Tenebris charge
    int moral_sway;             // [-100..+100] Moral alignment axis
    int threat_hp;              // [0..100] Threat vigor of active foe
    combat_stance_t stance;     // Active stance: warding, striking, equilibrium
    location_id_t location;     // Current Kingdom province
    bool is_running;            // Game loop active flag
    int turns_taken;            // Total turns / actions taken
    int foes_vanquished;        // Number of vanguard defeated
    int innocents_warded;       // Number of allies protected
} player_state_t;
```

All state variables are strictly bounded integers with zero dynamic allocations on the heap. This ensures deterministic cache locality, zero memory leaks, and instant serialization.

Chapter 40. 2. Universal Action Dispatcher (`game_process_command`)

Located in `src/sim/game.c`, `game_process_command()` acts as the singular entry point for all player intentions, whether emitted by the terminal CLI or the native GUI:

```
void game_process_command(player_state_t *state, const char *raw_input,
    char *out_msg, size_t msg_len);
```

40.1. Execution Pipeline:

1. **Input Sanitization**: Copies `raw_input` safely via `str_safe_copy()` and executes in-place whitespace trimming (`str_trim_inplace()`).
2. **Verb Tokenization**: Splits input into `command` and trailing `arg`.
3. **Action Dispatch**:
   - `ward`: Calls `handle_ward()`, modifying `shield_power` via `math_saturating_add()`, staggering active threats with `math_saturating_sub()`, and elevating moral virtue with `math_clamp_int()`.
   - `strike`: Calls `handle_strike()`, spending `sword_edge` via `math_saturating_sub()` to inflict heavy severance damage.
   - `balance`: Calls `handle_balance()`, simultaneously recovering both relics, parrying blows, and nudging moral sway toward 0 via `math_nudge_toward()`.
   - `stance`: Calls `handle_stance()`, shifting the tactical state machine.
   - `move`: Calls `handle_move()`, validating directional topology and spawning province encounters.
4. **Invariant Logging**: Writes a human-readable telemetry outcome into `out_msg` for presentation.

Chapter 41. 3. Diagnostic Self-Auditing (`game_audit_state`)

Following Block 4 of the OmniCode protocol, `src/sim/game.c` includes built-in diagnostic audit routines:

```
game_audit_report_t game_audit_state(const player_state_t *state);
int game_run_self_audit(void);
```

Every state boundary (shield power, sword edge, moral sway, stance, location) is checked against compile-time limits. If an invariant is violated, an audit report is generated with exact error messages, enabling automated continuous integration testing without relying on crude process crashes.

Chapter 42. Chapter 7: Layer 2 Presentation Viewport Code Tour

Source File: /home/professorseanex/Tenebris and Lux/docs/code/presentation_gui.adoc

Chapter 43. Overview

This document provides a comprehensive architectural and code walkthrough of the modular Layer 2 Presentation engine implemented in `include/gui/` and `src/gui/`. It details the decomposition of the graphical frontend into specialized sub-assemblies (`gui_palette`, `gui_layout`, `gui_widgets`, `gui_assets`, `gui_view2d`, `gui_hud`), the 2.5D diamond dimetric isometric projection engine with elevation prisms, the Asset Abstraction Layer for future artist drop-in assets, and the 60 FPS double-buffered X11 event loop.

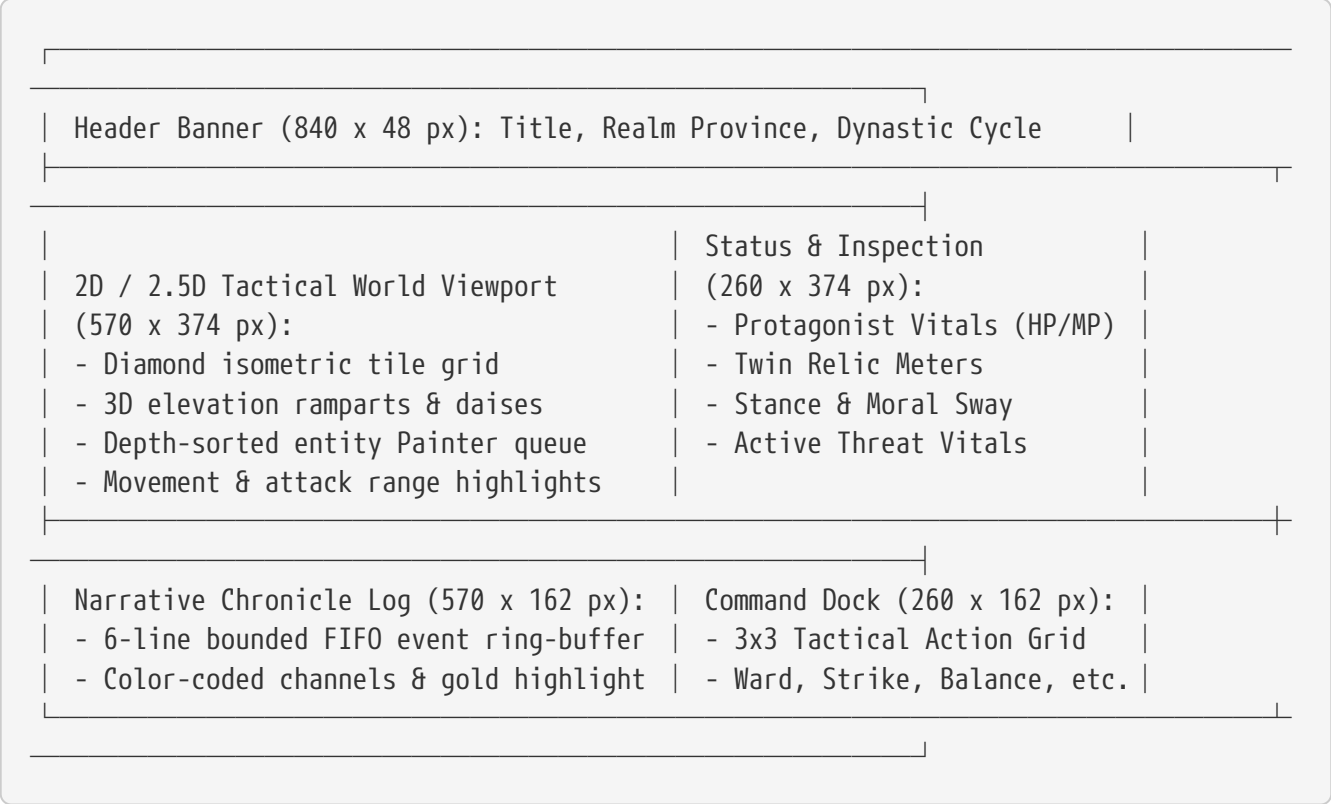
Chapter 44. 1. The Modular GUI Sub-Assembly Architecture

To prevent monolithic bloat and maintain strict separation of concerns, the presentation engine is partitioned into six single-responsibility sub-assemblies:

Sub-Assembly	Source Files	Architectural Responsibility
`gui_types`	`include/gui/gui_types.h`	Shared geometry, view modes, color, asset, and widget structs.
`gui_palette`	`gui_palette.c`, `gui_palette.h`	24-bit RGB / 32-bit ARGB colormap binding and 3D elevation shading.
`gui_layout`	`gui_layout.c`, `gui_layout.h`	Window partitioning into 5 disjoint panels with AABB non-overlap audits.
`gui_widgets`	`gui_widgets.c`, `gui_widgets.h`	Interactive buttons, case-insensitive hotkeys, and progress/relic meters.
`gui_assets`	`gui_assets.c`, `gui_assets.h`	Asset Abstraction Layer and procedural geometric vector art synthesizers.
`gui_view2d`	`gui_view2d.c`, `gui_view2d.h`	2D Flat / 2.5D Isometric tactical world viewport with elevation blocks.
`gui_hud`	`gui_hud.c`, `gui_hud.h`	Heads-Up Display: banner, vitals, twin relics, moral sway, and chronicle log.
`gui_core`	`gui.c`, `gui.h`	Master facade, X11 double-buffer backbuffer, and 60 FPS event dispatcher.

Chapter 45. 2. Desktop Panel Zoning & Layout Invariants

The presentation viewport (860×620 pixels) is partitioned into five distinct presentation zones designed with 10px outer margins and 8px inter-panel gutters:



Every panel bounding box is mathematically audited via `gui_layout_audit()` using Layer 0 `geom_rect_intersects()` and `geom_rect_within_bounds()` to guarantee zero visual collisions.

Chapter 46. 3. The 2.5D Tactical Isometric Engine

The game features dual visual projection modes toggleable at runtime via the `[M]` key: * `GUI_VIEW_2D_FLAT`: Orthographic top-down grid (38×38 px cells) for overworld travel. * `GUI_VIEW_25D_ISOMETRIC`: Diamond dimetric projection (48×24 px tiles) with vertical Z-axis elevation prisms (8 px per elevation level) for tactical battles.

46.1. Isometric Mathematical Formulation:

Given discrete 3D grid coordinates (gx, gy, gz), tile width W, tile height H, elevation scale E, and screen origin (X_0, Y_0):

```
screen_x = X_0 + (gx - gy) \cdot \frac{W}{2}
```

```
screen_y = Y_0 + (gx + gy) \cdot \frac{H}{2} - gz \cdot E
```

46.2. Unprojection Reciprocity:

Translating mouse clicks (sx, sy) on ground plane gz back to discrete grid (gx, gy):

```
gx = \left\lfloor \frac{sx - X_0}{W} + \frac{sy + gz \cdot E - Y_0}{H} + 0.5
\right\rfloor
```

```
gy = \left\lfloor \frac{sy + gz \cdot E - Y_0}{H} - \frac{sx - X_0}{W} + 0.5
\right\rfloor
```

46.3. 3D Prism Shading & Elevation Blocks:

When a tile has height $gz > 0$ (such as perimeter ramparts or altar daises), `gui_view2d.c` extrudes a 3-face shaded isometric cube: 1. **Front-Left Face**: Extruded downward by $gz \cdot E$, shaded at 75% brightness. 2. **Front-Right Face**: Extruded downward by $gz \cdot E$, shaded at 55% brightness (shadow). 3. **Top Diamond Face**: Rendered with ambient highlight at 115% brightness.

46.4. Painter’s Algorithm Depth Sorting:

Entities (Elijah, Katrina, Dream Eater, ShadowStalker) are inserted into a depth-sorted render queue keyed by:

```
depth\_key = (gx + gy) \cdot 1000 + gz \cdot 10 + layer\_bias
```

Rendering from lowest to highest depth key guarantees that foreground characters cleanly occlude background terrain and entities without depth buffer hardware dependencies.

Chapter 47. 4. The Asset Abstraction Layer

To ensure the game looks visually striking immediately while gracefully welcoming future hand-drawn sprites from artists, `gui_assets.c` implements an **Asset Abstraction Layer**:

1. **Asset Identity Registry**: Maps canonical asset IDs (`ASSET_HERO_ELIJAH`, `ASSET_ENEMY_DREAM_EATER`, `ASSET_TILE_COBBLESTONE`, `ASSET_RELIC_SHIELD_LUX`) to dimension specifications and target filepaths.
2. **Procedural Fallback Synthesizer**: In the absence of an external image, procedural vector generators draw character heraldry (e.g., Elijah's golden heater shield and ivory helm, Dream Eater's ethereal violet diamond nucleus with orbiting motes).
3. **Drop-in Artist Binding**: Calling `gui_assets_bind_external(id, path)` marks the asset as `ASSET_KIND_BITMAP`, routing draw calls to the spritesheet blitter without modifying simulation logic.

Chapter 48. 5. Verification Telemetry

The presentation stratum is validated via `tests/unit/test_gui_unit.c` across four ontological sections: * **Section 1 (Always Pass)**: Verifies default layout geometry, palette alpha opacity, and 9-button grid. * **Section 2 (Never Fail)**: Validates negative shading factor clamping, out-of-bounds click safety, and case-insensitive hotkey resolution. * **Section 3 (Assertion & Guarantee)**: Enforces AABB panel pairwise disjointness, projection reciprocity, and chronicle FIFO ring-buffer shift. * **Section 4 (Emergence)**: Probes X11 display availability and runs full self-audit pre-flight checks.

Chapter 49. Chapter 8: Layer 4 Verification & Test Suite Code Tour

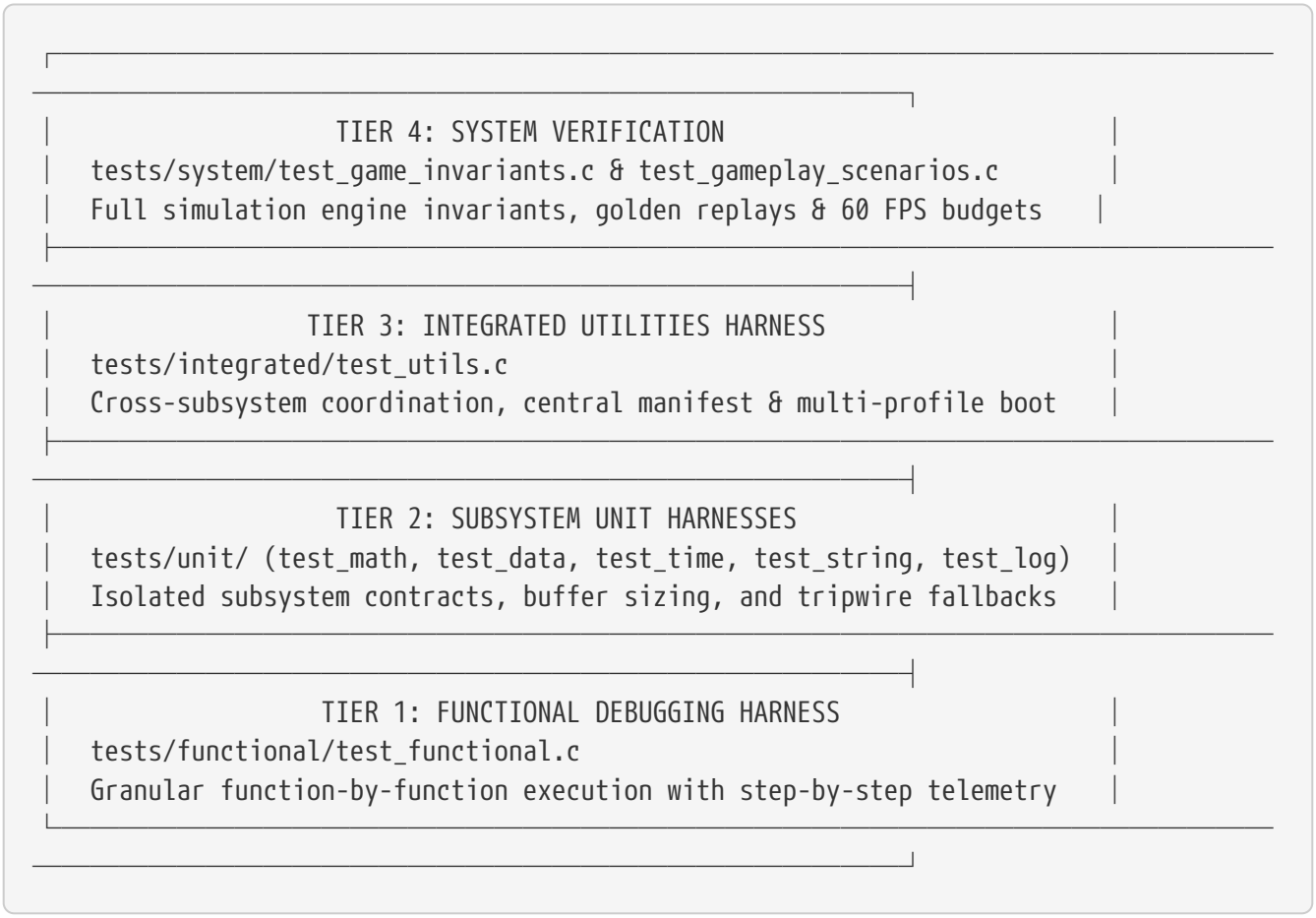
Source File: /home/professorseanex/Tenebris and Lux/docs/code/test_suite.adoc

Chapter 50. Overview

This document provides a comprehensive technical walkthrough of the automated game development verification architecture in **Tenebris and Lux**. Organized into strict hierarchical strata across `tests/`, the test suite mirrors the Layer 0-4 engine ontology and enforces four distinct ontological verification sections within every test harness:

1. **Always Pass (Nominal Golden Baseline)**: Verifies nominal parameters, default asset loading, and canonical execution paths.
2. **Never Fail (Fault-Tolerant Tripwires)**: Injects boundary breaches, NULL pointers, missing files, and corrupt states to prove that the engine never crashes or invokes undefined behavior.
3. **Assertion & Guarantee (Axiomatic Invariants)**: Mathematical guardrails, energy conservation, monotonic clocks, and DAG dependencies guaranteed by engine contracts.
4. **Emergence (Stochastic Stress & Chained Pipelines)**: Emergent combinatorial state spaces, 1,000 chaotic command fuzzing, bitwise golden replay determinism, and sub-50-microsecond simulation latency budgets.

Chapter 51. 1. The Four-Tier Testing Hierarchy



51.1. Tier 1: Functional Debugging (`tests/functional/`)

Designed specifically for interactive debugging and pinpointing anomalies in isolated functions. Emits explicit `[DEBUG-TRACE]` entries showing arguments before and after execution for each individual cog (e.g. `math_clamp_int`, `math_saturating_add`, `str_safe_copy`, `str_trim_inplace`, `json_get_string`, `combat_get_stance_name`).

51.2. Tier 2: Subsystem Unit Harnesses (`tests/unit/`)

Provides 1:1 test coverage for each dedicated Layer 0 utility subsystem folder: * **`test_math.c`**: Bounded arithmetic, saturating additions/subtractions, step-bounded equilibrium nudging, proportional ratio scaling, 2D discrete AABB hit-testing, and `data/math/default.json` profile loading. * **`test_data.c`**: Zero-heap filesystem I/O (`fs_utils.c`), canonical buffer sizing (`FS_BUFFER_SMALL_BYTES` = 1024 to `FS_BUFFER_MAX_CONFIG` = 65536), bounded file reading/writing, scoped JSON object querying, and array iteration. * **`test_time.c`**: Monotonic millisecond/microsecond clocks, POSIX nanosleep frame pacing, ISO 8601 timestamp generation ("YYYY-MM-DDTHH:MM:SSZ"), and 60 FPS / 120 FPS profile loading. * **`test_string.c`**: Buffer-safe bounded copying, in-place whitespace trimming, case-insensitive equality/substring matching, parent directory extraction, and duplicate-slash immune path joining. * **`test_log.c`**: Forensic telemetry, graded severity filtering, categorized logging channels, in-memory static ring buffer, and emergency monolithic code page fallbacks. *

`test_debug.c`: Interactive state debugger, frozen snapshot ring buffer, FNV-1a checksum determinism, time-travel state rewind, state diffing, dynamic breakpoints, and emergent chaotic stress.

51.3. Tier 3: Integrated Utilities Harness (`tests/integrated/`)

- `test_utils.c`: Validates cross-subsystem coordination of all Layer 0 utilities. Exercises the central manifest registry (`manifest.c`), validates upstream DAG dependency resolution, enforces diagnostic escalation policies (WARNING → `PREFLIGHT_STATUS_WARNINGS`, CRITICAL → `PREFLIGHT_STATUS_DOWNGRADED`, FATAL → `PREFLIGHT_STATUS_HALT_FATAL`), and benchmarks multi-profile synchronous boot latency (< 5 milliseconds).

51.4. Tier 4: System Verification (`tests/system/`)

- `test_game_invariants.c`: Validates Layer 1 simulation engine state invariants: twin relic power bounds ($[0, 100]$), moral sway continuum boundaries ($[-100, +100]$), protagonist name customization, and bidirectional provincial map topology.
- `test_gameplay_scenarios.c`: Simulates end-to-end tactical combat routes (Righteous Warden and Umbral Striker paths), 15-step golden deterministic replay parity, 1,000 chaotic command fuzzing for softlock immunity, and 10,000-turn simulation latency benchmarking (< 50 microseconds per turn).

Chapter 52. 2. The Four Ontological Testing Sections

Every test harness in the repository implements the four-section ontological model:

Section	Category	Mission & Verification Scope
Section 1	**Always Pass**	Valid inputs, nominal parameters, default configuration loading, standard state initialization, well-formed JSON, normal gameplay actions.
Section 2	**Never Fail**	Inviolability under error. Injects NULL pointers, zero capacities, missing files, corrupted JSON, and unknown stances. Proves tripwire activation and safe fallback without OS traps.
Section 3	**Assertion & Guarantee**	Axiomatic mathematical laws and structural contracts: bounded clamping, non-decreasing monotonic clocks, strict null-termination under truncation, and deterministic stance completeness.
Section 4	**Emergence**	Higher-order systemic behaviors: 1,000 pseudo-random fuzzing iterations, spatial grid overlap emergence, golden replay bitwise equality across runs, and < 50 us latency budgets.

Chapter 53. 3. CMake & CTest Test Targets

The build configuration registers twelve automated test targets within `CMakeLists.txt`:

```
# CLI Interface Contract Verification Tests
add_test(NAME cli_help_test COMMAND tenebris-lux --help)
add_test(NAME cli_version_test COMMAND tenebris-lux --version)
add_test(NAME subsystem_audit_test COMMAND tenebris-lux --audit)

# Tier 1: Functional Debugging Harness
add_test(NAME functional_cogs_test COMMAND test_functional)

# Tier 2: Subsystem Unit Harnesses (Isolated Subsystems)
add_test(NAME unit_math_test COMMAND test_math)
add_test(NAME unit_data_test COMMAND test_data)
add_test(NAME unit_time_test COMMAND test_time)
add_test(NAME unit_string_test COMMAND test_string)
add_test(NAME unit_log_test COMMAND test_log)
add_test(NAME unit_debug_test COMMAND test_debug)

# Tier 3: Integrated Utilities Harness (Multi-Subsystem Coordination)
add_test(NAME integrated_utils_test COMMAND test_utils)

# Tier 4: System Verification Harnesses (Full Game Simulation)
add_test(NAME system_game_invariants_test COMMAND test_game_invariants)
add_test(NAME system_gameplay_scenarios_test COMMAND test_gameplay_scenarios)
```

Chapter 54. 4. Execution & Verification Commands

Developers and CI/CD pipelines can invoke the complete suite or isolate specific tiers:

```
# Compile all binaries and test targets under strict zero-warning policy
cmake --build build --config Debug

# Execute all 13 automated verification suites
ctest --test-dir build -C Debug --output-on-failure

# Run individual tier harnesses directly for step-by-step debugging
./build/test_functional         # Granular cog trace output
./build/test_math               # Math & geometry unit verification
./build/test_data               # Filesystem & JSON unit verification
./build/test_time               # Timing & clock unit verification
./build/test_string             # Safe string unit verification
./build/test_log                # Forensic telemetry unit verification
./build/test_debug              # State debugger & rewind unit verification
./build/test_utils              # Integrated utilities verification
./build/test_game_invariants    # Simulation state invariants
./build/test_gameplay_scenarios # Gameplay scenarios & latency benchmark
```

Chapter 55. Chapter 9: Technical Specification: Data-Driven Architecture & Manifest System

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/data_driven_architecture.adoc

Chapter 56. Abstract & Architectural Mandate

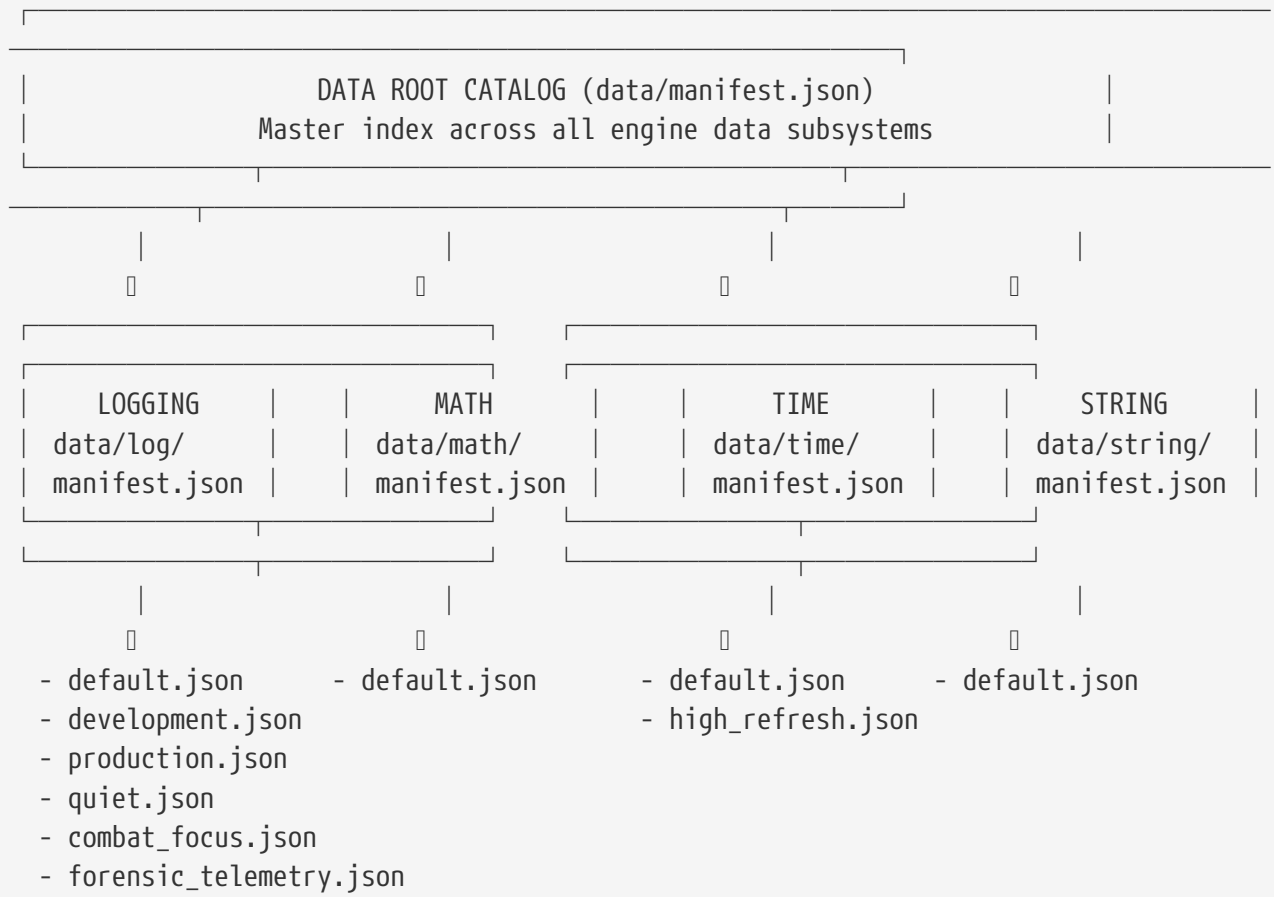
This specification establishes the **Data-Driven Subsystem** for **Tenebris and Lux**. Transitioning from hardcoded compile-time values to externalized, validated data files, this architecture enforces the **"Manifest as Main Menu" and "Files as Settings"** ontological pattern. This pattern enables live configuration hot-swapping, deterministic testing, and seamless integration between C11 Layer 0 engine utilities and high-level tools. It encompasses logging profiles (`data/log/`), mathematical continuum boundaries (`data/math/`), frame pacing budgets (`data/time/`), and lexical token disciplines (`data/string/`).

Chapter 57. 1. System Philosophy: Manifest as Main Menu

57.1. 1.1 The Menu-to-Settings Ontological Pattern

Traditional game configuration often conflates cataloguing with concrete configuration. Under the **Tenebris and Lux** architecture, a clean structural boundary is maintained:

- **The Manifest (`manifest.json`)**: Operates as the **"Main Menu"**. It catalogs, describes, indexes, and exposes selectable configuration entries (`menu_items`). It enforces invariants governing which settings are available, provides default selections, and maps hotkey shortcuts.
- **The Data Files (`*.json`)**: Operate as the concrete **"Settings"**. Each file represents a fully-formed, isolated configuration state conforming directly to the underlying engine structures (e.g., `log_config_t`, `math_config_t`, `time_config_t`, `string_config_t`).



57.2. 1.2 OmniCode 5-Block Conformance in JSON Serialization

In accordance with the repository standard, every JSON data asset strictly adheres to the **Universal 5-Block Lifecycle Protocol**:

1. **Block 1:** `_metadata`: Identity tags (`@project`, `@file`, `@id`, `@version`, `@author`, `@invariants`).
2. **Block 2:** `_types`: Schema specification, target C struct bindings, enum sets.
3. **Block 3:** `menu` / `profile` / `setting`: Operational payload containing menu entries or profile parameters.
4. **Block 4:** `_audit`: Diagnostic limits, rotation caps, and memory footprint contracts (`ALLOC_ZERO_HEAP`).
5. **Block 5:** `_navigation`: Upstream parent manifest links and fallback targets.

Chapter 58. 2. Active Data-Driven Subsystems

58.1. 2.1 Logging & Telemetry (`data/log/`)

Defines six distinct configuration profiles (`default`, `development`, `production`, `forensic_telemetry`, `quiet`, `combat_focus`) controlling min severity, console ANSI echoing, file rotation limits, and telemetry tracking categories.

58.2. 2.2 Mathematical Continuum (`data/math/`)

Defines canonical bounds and continuum limits governing game metaphysics:

- **Resonance Limits**: `[0, 100]` strictly preventing relic over-saturation or underflow.
- **Moral Sway Continuum**: Symmetric `[-100, +100]` with equilibrium anchor at `0`.
- **Equilibrium Stance Band**: `[-25, +25]` threshold for dual relic synergy.
- **Overload / Eclipse Thresholds**: `+75` (Radiance) and `-75` (Abyssal Eclipse).
- **Viewport Dimensions**: Default desktop presentation bounds (`800 x 600`).

58.3. 2.3 Monotonic Timing & Frame Pacing (`data/time/`)

Defines latency budgets and frame sleep targets:

- `default.json`: 60 FPS profile (`16.666667 ms` frame budget, `50.0 us` simulation turn budget).
- `high_refresh.json`: 120 FPS profile (`8.333333 ms` frame budget, `25.0 us` simulation turn budget).

58.4. 2.4 Lexical Buffers & Token Dictionary (`data/string/`)

Defines textual safety limits and canonical identifiers:

- **Buffer Capacities**: Max path (512 B), max name (64 B), max command (128 B), line discipline (80 chars).
- **Stance Tokens**: `NEUTRAL`, `WARDING`, `STRIKING`, `EQUILIBRIUM`.
- **Action Verbs**: `help`, `version`, `audit`, `cli`, `gui`, `barrio`, `cuttatio`, `duo`, `stance`, `journey`, `inspect`, `name`, `quit`.

Chapter 59. 3. The Fallback Tripwire Protocol

A central invariant of the data-driven architecture is **Fault-Tolerant Determinism**:

1. **Attempt Data Load**: Subsystems first attempt to deserialize settings from their respective `data/` JSON file.
2. **Tripwire Detection**: If the path does not exist, file permissions fail, or JSON formatting is invalid, the engine detects this condition immediately without crashing or memory corruption.
3. **Hardcoded Fallback Activation**: The subsystem loads its compile-time fallback baseline (e.g. `math_get_default_config()`, `time_get_default_config()`, `string_get_default_config()`, `log_get_default_config()`).
4. **Diagnostic Telemetry**: A warning trace is emitted to the diagnostic log detailing the fallback event and preserving system stability.

Chapter 60. 4. Verification & Diagnostic Testing

All data files in `data/` and its subsystem folders are validated for syntactic correctness, schema compliance, and referential integrity by CTest suites (`test_utils`, `test_log`, `test_game_invariants`) and developer tooling (`tl_tools`).